Appendix: The Attacks and Tools, Explained

You have met these ideas in lectures as theory. The point of this appendix is to connect that theory to practice: to name each attack family, explain why it works, and say which tools and steps you would use. It does not give you code or your own instance's values. Think of it as the briefing a mentor would give before you sit down at the keyboard.

A note that runs through all of it: SmartLock sends its handshake and records over plain TCP in a line-based text format. That is not a separate flaw to exploit; it is simply the **capture surface**. It is what lets you see the exchange at all, the same way you would observe a real protocol in Wireshark before reasoning about it.

1. Reading the wire: capture first

Before any attack, you watch a normal session. On your capture interface (the host-only link between the lock VM and the hub VM, or `lo` for a single-host smoke test), record the TCP stream and read it.

- **tcpdump / tshark** capture packets to a `.pcap` file from the command line.
- **Wireshark** opens that file (or captures live). Its **Follow TCP Stream** view reassembles the connection so you can read the whole handshake as text: the `HELLO`, the `SUITE`, the `CERT`, the `PAIR`, the `DH` and `DHCLIENT` values, and the `MSG` records.

```
sudo tcpdump -i <iface> -w session.pcap tcp port <PORT>
```

Everything that follows starts from one such capture. Save a `.pcap` per finding; those captures are your Part A evidence.

Concepts to revisit: the TCP three-way handshake and stream reassembly; the idea of a **man-in the-middle** position (on-path) versus a purely **passive** eavesdropper (off-path); and the difference between reading traffic and modifying it.

2. Stage 1: Negotiation and downgrade

The idea. When two parties agree which cipher to use, that agreement must itself be protected. If an attacker can edit the list of offered ciphers, or if one of the offered options is "no encryption", the attacker can push both sides down to the weakest choice. This is a **downgrade attack**. You have seen the real-world versions in lecture: attacks such as FREAK and Logjam forced strong TLS sessions down to deliberately weak, export-grade cryptography by tampering with the negotiation.

Why it applies here. The hub's offered-suite list travels in clear, nothing binds it (no signature or MAC over the negotiation), and one of the named suites is `NULL`, meaning no record encryption at all. An on-path attacker who rewrites the hub's `HELLO` so that it offers only `NULL` leaves the lock with no strong option to pick. Every record afterwards, including the greeting that carries the flag, is then plaintext on the wire.

How you would do it. You need an on-path position so you can edit a message in flight:

- With three VMs, put yourself between hub and lock with ARP spoofing using **ettercap** or **bettercap**, enable IP forwarding, and rewrite the `HELLO` line as it passes.

- With two VMs, model the on-path attacker with a small local **relay**: a TCP proxy that the hub connects to, which forwards to the real lock and rewrites the `HELLO` (and, for Stage 2, the `CERT`) line. **mitmproxy** in a reverse mode with your own addon, or a short relay of your own, both work. Point the hub at it with `hub_client` 's optional port argument (see the Lab Setup Guide). **Scapy** can also craft or edit frames.

What to show. The rewritten negotiation in a capture, and a record you can now read in clear. The property that fails is **confidentiality** (and the integrity of the channel, since nothing detected the tampering).

3. Stage 2: Authentication (impersonation and the PIN)

Two distinct weaknesses live in this stage.

3a. Impersonation / man in the middle

The idea. A certificate is only worth as much as the checking done on it. If the receiver never verifies the certificate against a trust anchor, anyone can present any certificate. Combined with an **unauthenticated** key exchange (one where the public values are not signed), this lets an attacker sit in the middle: it completes one key exchange with the hub and another with the lock, and relays between them, reading and even altering everything. Revisit the lecture material on digital certificates, trust anchors, and why a self-signed certificate that nobody validates proves nothing.

Why it applies here. The lock's certificate is self-signed and the hub never validates it, and the Diffie-Hellman values are not signed. A man in the middle presents its own certificate and its own Diffie-Hellman value to the hub; the hub cannot tell the difference and shares a key with the attacker instead of the lock.

How you would do it. The same on-path position as Stage 1 (ARP spoofing with ettercap/bettercap, or a relay). The relay substitutes the `CERT` and the `DH` values and keeps a key with each side. The property that fails is **authentication**.

3b. Recovering the pairing PIN

The idea. A password or PIN should never be stored or transmitted as a bare, fast, unsalted hash. MD5 is fast and unsalted here, so an attacker who captures the hash can simply try candidate PINs, hash each one, and compare. This is an offline **dictionary / brute-force** attack. Because the PIN has a known shape, a dictionary word followed by three digits, the search space is tiny (a few thousand candidates).

How you would do it. Take the `pinhash` value from a captured `PAIR` message and feed it to a cracker with a wordlist and a three-digit mask:

```
# MD5 (-m 0), hybrid wordlist + mask (-a 6), three trailing digits
hashcat -m 0 -a 6 captured_pin.md5 words.txt ?d?d?d
```

John the Ripper does the same with rules. The property that fails is the confidentiality of the **credential**. Recovering the hash without cracking it is not enough: show the recovered PIN.

Concepts to revisit: cryptographic hashes and preimage resistance; why speed and the lack of a salt make a hash crackable; the roles of dictionaries, masks, and rules.

4. Stage 3: Keying (discrete log, no KDF, static exponent)

This stage packs three separate lessons.

4a. A small group means the discrete log is solvable. Diffie-Hellman's security rests on the **discrete logarithm problem**: given `p`, `g`, and `B = g^b mod p`, recovering `b` should be infeasible. That is only true when `p` is large (2048 bits or more in practice). SmartLock uses a prime of about 40 bits, which is small enough to solve in seconds.

The standard method is **baby-step giant-step**, a meet-in-the-middle search. Write the unknown exponent as `b = i·m + j` with `m ≈ sqrt(p)`. Build a table of `g^j` for every small `j` (the "baby steps"), then repeatedly multiply `B` by `g^(-m)` (the "giant steps") and look for a match in the table. A match gives `i` and `j`, hence `b`. The work is about `sqrt(p)` instead of `p`, which is why a 40-bit prime falls in moments and a 2048-bit prime never would. You can implement this in a short C++ program using big-integer-free 64-bit arithmetic for this group, or use a library.

Once you have `b`, combine it with the captured hub value `A` to reach the same shared secret the two parties used, turn it into the record key the way the protocol does, and decrypt the greeting to read the flag.

4b. No key derivation function. Even with a large group, using the raw shared secret directly as an AES key is a separate mistake. A proper design runs the secret through a **key derivation function** (a KDF such as HKDF) to produce a uniformly random key bound to the session context. Using the secret directly leaks structure and skips that binding. Note this as its own weakness, distinct from the small group.

4c. A static device exponent means no forward secrecy. The lock reuses the same secret exponent `b` for every session. **Forward secrecy** is the property that compromising long-term material does not expose past sessions; it comes from using fresh, **ephemeral** exponents each time. A static `b` destroys it: recover `b` once and you hold the key to every past and future session, not just the one you captured.

The properties that fail are **keying** (confidentiality of the key) and **forward secrecy**.

5. Stage 4: Message protection (nonce reuse)

The idea. AES-GCM is an authenticated stream-style cipher: internally it generates a keystream from the key and the nonce and XORs it with the plaintext, then computes an authentication tag. The absolute rule is that a given **(key, nonce) pair must never be reused**. If it is, two ciphertexts share the same keystream, and XORing them cancels the keystream entirely:

```
C1 = P1 XOR KS
C2 = P2 XOR KS
C1 XOR C2 = P1 XOR P2
```

This is the classic **two-time pad**. If you know or can guess part of one plaintext (and you do: the greeting begins with a fixed, known prefix), you recover the matching bytes of the other. Worse, nonce reuse in GCM also leaks enough to **forge** valid tags (the so-called forbidden attack), breaking integrity as well as confidentiality.

Why it applies here. The lock uses a fixed nonce for every record under the same key, and transmits it, so the reuse is plain to see in a capture.

How you would do it. Take two records from one session (for example the greeting and a `STATUS`

reply), confirm they share the nonce, XOR their ciphertext bodies, and use the known greeting prefix to read the other plaintext. A few lines of C++ or a CyberChef recipe suffice. The properties that fail are **confidentiality** and **integrity** of the records.

Concepts to revisit: stream ciphers and the two-time-pad problem; what a nonce is and why uniqueness, not secrecy, is its job; AEAD and the meaning of the authentication tag.

6. Stage 5: Teardown (replay)

The idea. A message can be authentic and still be dangerous if it can be **replayed**. Without a freshness mechanism, a per-session sequence number, a counter, or a timestamp bound into each record, an attacker who captures a valid command can resend it later and have it accepted again. For a door lock this is the headline result: a captured `UNLOCK` becomes a permanent skeleton key.

Why it applies here. Records carry no sequence number, the lock keeps no anti-replay state, and there is no authenticated close. On top of that, the static device exponent and static nonce from Stage 3 and Stage 4 mean a replayed handshake reproduces the very same session key, so a replayed `UNLOCK` record still decrypts and still opens the door. The unlock counter lasts across sessions, so you can watch it climb as you replay.

How you would do it. Capture a full client-side exchange and the `UNLOCK` record, then reconnect and resend the recorded lines from a short socket program. Replaying the raw packets with **tcpreplay** does not work on its own, because a new TCP connection has new sequence numbers. Show the counter increasing. The properties that fail are **integrity** and **availability** (the owner can no longer trust that a command happened once and only once).

Concepts to revisit: replay attacks and freshness; nonces and counters as anti-replay state; sequence numbers; truncation attacks and why a close must be authenticated.

7. Tool reference at a glance

Tool	What it is for
Wireshark / tshark / tcpdump	Capture and read the TCP stream; Follow TCP Stream reassembles the handshake.
ettercap / bettercap	ARP-spoofing man in the middle on a shared LAN, to get an on-path position.
mitmproxy	A programmable proxy; in a reverse mode it models an on-path attacker between hub and lock.
Scapy	Craft, read, and edit packets and frames from Python, for custom interception.
hashcat / John the Ripper	Offline cracking of the captured MD5 pairing hash with a wordlist and a digit mask.
tcpreplay	Replay captured packets back onto the wire. It cannot complete a new TCP connection by itself, so use a socket client for the Stage 5 replay.
openssl / CyberChef	Quick cryptographic operations: hashing, base64, XOR, and sanity checks.
your own C++ programs	The discrete log, the nonce-reuse XOR, and the replay client; the heart of the work.

8. Ethics and scope

Every technique here is powerful and is taught so you can defend against it. Use it **only** inside your isolated lab, only between your own virtual machines, and never against any system you do not own or are not authorised to test. Keep the lab network host-only or internal, never bridged to the campus network or the internet, and confirm this in your report's "Scope and ethics" paragraph. The skill being assessed is judgement as much as technique.